Author Response — Full Version

Quantum Computing and Data Processing for Frequent Itemset Mining

August 2026

This document collects the authors' full technical clarifications prepared in response to the reviews of this work. Only the authors' own responses are reproduced; no reviewer text is included. The short labels (e.g. R2W3D3) are the identifiers used in the submitted response letter. Bracketed numbers refer to the reference list of the manuscript; [35] is the full version of this work.

Reviewer 1

R1W1 — Error measurements and logical-qubit footprint

R1W1: Sorry. Due to space limits, following [16,17], we prioritize runtime and scalability in the main paper; error measurements and qubit-footprint analysis on Amazon Braket QPU and IBM Qiskit are in Appendix D of our full arXiv version [35] (pp.18-19). For qubit footprint, QFM has lower circuit depth without requiring more logical qubits than qARM (Table VIII, p.18). For error measurements, on IonQ Forte 1, all 33 QPU runs achieve 100% correctness (Table IX, p.19); under Qiskit Aer noise, correctness is 100%/99.7%/99.0%/92.7% at two-qubit error rates 0/0.006/0.010/0.015 (Table X, p.19). Footnote 28 (p.19) further explains QIL's error handling: returned itemsets are verified against retained parent transaction-indicator vectors to reject false positives, while repeated extraction with a larger budget mitigates missed itemsets.

R1W2D1 — The Amazon Braket runs are QPU executions, not simulations

R1W2D1: Our Amazon Braket experiments are quantum implementations on QPU hardware (IonQ Forte 1), not simulations. QFM includes theoretical analysis, IBM Qiskit, and Amazon Braket QPU implementation. Traditional theoretical quantum algorithms [49,52] provide only complexity analysis, while qARM [54,55] uses Qiskit; none reports Amazon quantum implementation.

R1W3 — Determinism of the reported output, and relation to sampling-based FIM

R1W3: QFM is not designed to gain efficiency by sampling the database; instead, QIL uses candidate verification, boundary estimation, and dynamic deduplication to extract complete patterns (Sec. IV-C, p.8; Footnote 28 of [35], p.19). By contrast, [A,B] use database sampling to reduce mining cost. [A] provides ε-close frequent itemset approximation with probability

1−δ, while [B] handles sampling misses through further candidate generation and another full database pass. On Nursery, our implementation of [B] takes 0.069-0.147 s at σ=10-50% (Table III) and returns exactly the deterministic baselines' frequent sets in every run, with its verifying database pass alone accounting for 65-83% of that time.

[A] M. Riondato and E. Upfal. Efficient discovery of association rules and frequent itemsets through sampling with tight performance guarantees.

[B] H. Toivonen. Sampling large databases for association rules.

R1D2 — The O(NM) loading term

R1D2: The O(NM) loading is a one-time input cost to read all data items and transactions, also required by Apriori, FP-growth, and Eclat, i.e., not QFM-specific. QFM's mining cost O(Q_hat{k}(log B_hat{k}+log M)) may exceed NM under large iterative mining workload, i.e., loading cost not dominant; meanwhile, Apriori and FP-growth grow further to O(NM2^N) and O(M2^N) (Table VII of [35], p.17).

R1-9 — Reproducibility of the released code

R1-9: All simulations are reproducible with our code. Amazon Braket QPU runs require only a Braket-enabled AWS account and are not restricted by author-controlled access.

Reviewer 2

R2W1D1 — Completeness of L1 under bounded-error amplitude estimation

R2W1D1: Due to space limits, Appendix A of our full arXiv version [35] (Footnote 17, p.15) clarifies that. QAE prunes a singleton only when its certified interval lies entirely below σ; otherwise QPR resolves its exact support by popcounting the materialized transaction-indicator vector before L1 is finalized (as traditional algorithms do), so L1 is complete.

R2W3D3 — How the reported time in Tables II, III and V is obtained

R2W3D3: Because current Amazon Braket QPUs cannot accommodate larger datasets, QFM time is returned by Qiskit's duration attribute according to [61-64], using hardware-specific timing information, including target-specific instruction durations, gate/measurement times, code-cycle times, and reaction times, under surface-code superconducting, Google/KTH large-scale superconducting-qubit, and Microsoft Azure AQRE models [61-64]. IBM Qiskit includes in-circuit state preparation, quantum gate execution, scheduled delays, and measurement, but excludes compilation, shots, and classical pre-/post-processing.

Reviewer 3

R3W1C1 — Dataset selection

R3W1C1: We select datasets adopted in prior studies [28,36,47,59,60], covering biology, board games, census records, traffic accidents, bike-sharing demand, and image data with a wide range of transaction/item counts. We especially include larger datasets to test heavier workloads (Fig. 3, p.11) and circuit resource use (Table IV, p.11).

R3W2C2 — Why qARM is slower than sequential miners on Mushroom

R3W2C2: The reason is that qARM is only a partial quantum algorithm. qARM has traditional (non-quantum) level-wise candidate generation, and it uses quantum computing only for support counting on given candidate itemsets (Footnote 7, p.3). On Mushroom, dense transactions generate many frequent items, so its level-wise join significantly increases the candidate space at every level, and each candidate still requires a support check. FP-Growth avoids explicit candidate generation via FP-tree compression, explaining its lower runtime.

R3W3C3 — Repeated runs and run-to-run variability

R3W3C3: In experiments, QFM is verified to produce the same solutions as Eclat, FP-growth, HAMM, and CICLAD under each dataset/parameter setting, i.e., no randomized search and therefore will not generate various solutions. PHA-HUIM, in contrast, is a population-based stochastic heuristic.

R3-Overall — Summary

R3-Overall: We thank the reviewer for the careful reading and constructive suggestions; the clarifications above address dataset choice (C1), the qARM behavior on Mushroom (C2), and statistical reporting (C3).

Reviewer 4

R4W1 — The O(SM) classical synthesis cost

R4W1: The S retained vectors are synthesized only once into the level-k load oracle at O(SM) classical cost and reused across all QIL calls, so O(SM) is not multiplied by the QIL call count. With S=|L_k|, the synthesis cost across levels is O(M sum_k |L_k|). Including this cost does not change the order of the classical overhead, because Footnote 21 of our full arXiv version [35] (p.16) already gives an inter-level materialization term for retained vectors of the same order.

R4W2 — Which baseline implementations are used

R4W2: Following common practice, we directly use the implementations released in [48,57] for PHA-HUIM and CICLAD and the renowned SPMF [60] for Eclat and FP-growth. All are publicly available via GitHub or SPMF library.

R4W3D4 — The role of [61]-[64], and the target architecture

R4W3D4: Sec. V.A (p.10) states, "Following [61-64], we evaluate QFM using the following metrics: time, logical depth, and logical gate count." Here, [61-64] only provide runtime estimation for evaluation, not QFM implementation or system architecture, with runtime based on hardware-specific timing information, including gate/measurement times, code-cycle times, and reaction times, under surface-code superconducting, Google/KTH large-scale superconducting-qubit, and Microsoft Azure AQRE models [61-64]. QFM's system architecture comprises its design: QPR, QSV, and QIL (Sec. IV, p.5), built on Bit-Vector Qubit Encoding, MAC Superposition, and Bit-Parallel Threshold Marking (pp.4-5), not following [61-64].

R4W4 — Data movement and remote-service I/O

R4W4: Thanks. Data movement in Sec. V.B refers to device-to-device memory transfers within PHA-HUIM's iterative GPU computation (L838-843 of the code provided by [48]), not remote service I/O. Local IBM Qiskit does not have remote job submission or queue latency, and our GPU experiments likewise run locally without such service overhead.

R4D1D2 — GPU hardware and software environment

R4D1D2: NVIDIA GeForce RTX 3090 GPUs (24 GB, CUDA 12.2) for PHA-HUIM; Qiskit 1.1.1 with Qiskit Aer 0.14.2 on Python 3.12 (NumPy 2.0.0).

R4D3 — Comparison with PHA-HUIM's stated complexity

R4D3: Please note that PHA-HUIM's complexity [48] is derived under a fixed heuristic search budget, i.e., the search stops after this budget rather than after all patterns are found. We remove this stopping condition and continue the same GPU-parallel search with support-based filtering (Footnote 15, p.10) to find all frequent itemsets (for fair comparison with other baselines), so its runtime need not follow O(maxlen).

Reviewer 5

R5W1Q1 — How the reported time is obtained, and a hardware-agnostic comparison

R5W1Q1: Because current Amazon Braket QPUs cannot accommodate larger datasets, QFM time in Tables II, III and V is returned by Qiskit's duration attribute according to [61-64], using hardware-specific timing information, including target-specific instruction durations, gate/measurement times, code-cycle times, and reaction times, under surface-code superconducting, Google/KTH large-scale superconducting-qubit, and Microsoft Azure AQRE models [61-64]. IBM Qiskit includes in-circuit state preparation, quantum gate execution, scheduled delays, and measurement, but excludes compilation, shots, and classical pre-/post-processing. As suggested, we further add oracle calls to Table IV as a hardware-agnostic comparison: 160/280 for QFM versus 9,320/29,960 for qARM on Bike/Skin.

R5W2Q2 — Peak logical-qubit footprint and space complexity

R5W2Q2: Due to space limits, QFM's space complexity is in Theorem B.1 and Footnote 23 of our full arXiv version [35] (p.17). Theorem B.1 (p.17) gives O(M+log|C_{k+1}|), and hence O(M+min{2log|L_k|,(k+1)logN}), which counts active workspace qubits; with level-wise eviction, Footnote 23 (p.17) gives O(M max_k|L_k|) classical bits for retained transaction-indicator vectors. By contrast, qARM provides no space complexity analysis, and its experimental implementation [55] is limited to databases up to 4×4.

R5W3Q4 — The parameter regime in which QFM is favorable

R5W3Q4: Due to space limits, QFM's complexity advantage and favorable regime are in Table VII and Corollary B.1 of [35] (pp.16-17). 1) Table VII shows that FP-growth has an M-multiplicative mining cost, while QFM has only log M dependence after O(NM) preprocessing (required by all schemes). 2) Corollary B.1 bounds QFM by O(NM+\hat{k}dB(log dB+log M)), where B=max_k|L_k| and d is the maximum number of frequent itemsets sharing a prefix. Thus, QFM is asymptotically better than FP-growth for large M and bounded candidate growth (B,d). The gap further increases with smaller level-wise frequent sets and fewer frequent itemsets sharing the same prefix, reducing B and d (Appendix B of [35], p.16).

R5W4 — The classical/quantum boundary, compared with qARM

R5W4: Due to space limits, QFM's advantage over qARM is in Table VII of [35] (p.17). qARM retains classical join-and-prune candidate generation (Sec. II, p.3), whereas QFM moves candidate preparation, support verification, and itemset extraction into quantum circuits. Specifically, 1) candidate preparation: classical join-and-prune (qARM) vs MAC superposition (QFM); 2) support verification: ε-approximate support vs exact threshold marking (Footnote 7, p.3; Table VII of [35], p.17); and 3) itemset listing: quantum support checking over classically generated candidates vs QIL amplification/extraction (Footnote 7, p.3).

R5W5Q6 — Noise-model evaluation

R5W5Q6: Due to space limits, reliability under real hardware and controlled noise is in Tables IX-X of [35] (p.19). QFM achieves 100% correctness across all Amazon Braket IonQ Forte 1 runs (Table IX) and remains 99.0% correct under Qiskit Aer at 1.0% two-qubit gate error (Table X). As suggested, we can additionally provide Aer noise-model simulations of the threshold-decision subcircuit across M and σ.

R5Q3 — Cost of the load oracle and inter-level cache update

R5Q3: Due to space limits, inter-level cache update and materialization cost are in Footnotes 18 and 21 of [35] (pp.15-16). 1) QFM does not require hardware QRAM. Footnote 13 (p.9) specifies O_load as a reversible lookup over the materialized level-k cache with O(log|L_k|)-depth addressing. Synthesizing it from the known cache uses O(M|L_k|) gates. 2) The cache is updated incrementally, not rebuilt from the database (Footnote 18, p.15). Each verified itemset's transaction vector is derived from its parents and retained for the next level, with O(M|L_{k+1}|) classical bit operations. 3) The cost of O_load enters Theorem IV.3 through each QSV call in Lemma IV.2 (p.9) and is absorbed into O(log \hat B_k) after summing over levels (p.10). If charged as sequential bit-level work, inter-level materialization adds O(M sum_k |L_k|) (Footnote 21, p.16).

R5W6Q7 — Additional state-of-the-art C baselines

R5W6Q7: To avoid reimplementation bias and preserve the authors' optimizations, we use the C/C++ code released by the authors of [48,57] for PHA-HUIM and CICLAD, and the widely used SPMF Java implementations [60] for Eclat and FP-growth. As suggested, we add Uno's LCM and Borgelt's FP-growth and dEclat, all in C; every output was verified to contain exactly the same number of itemsets as an exact reference miner, with no mismatch. Regarding the high-utility miners, HAMM and PHA-HUIM are included because they are the closest available GPU/utility-oriented systems, and we are happy to move them to a secondary comparison.

Table R1 reports the three added C miners over sigma = 30-50% on all four datasets, the favorable regime that our own analysis delimits (R5W3Q4). Across these twelve settings the frequent-itemset population |L| ranges from 1 to 2,733, LCM takes 0.0034-0.0347 s, FP-growth 0.0037-0.0253 s and dEclat 0.0039-0.0266 s, and QFM's tabled time stays 23-110x below the fastest of the three in every setting. The margin is widest on Car and Tic-Tac-Toe, where |L| is at most 25, and narrowest on Mushroom, where |L| reaches 2,733; the advantage therefore tracks the bounded candidate growth condition B = max_k |L_k| of Corollary B.1 (R5W3Q4).

Table R1. Added C-implementation baselines over the favorable regime (sigma = 30-50%, tau = 1). Each entry is the mean of five runs on the same server; ranges are over the three sigma values. All outputs count-verified against an exact reference. QFM's column is the time reported in Tables II-III (see R2W3D3 for how it is obtained).

Dataset	|L|	LCM (C)	FP-growth (C)	dEclat (C)	QFM advantage
Tic-Tac-Toe	1-25	0.0034-0.0050	0.0037-0.0050	0.0039-0.0054	91-110x
Nursery	2-18	0.0170-0.0221	0.0167-0.0193	0.0167-0.0196	99-105x
Car	1-12	0.0040-0.0042	0.0045-0.0046	0.0044-0.0047	85-94x
Mushroom	153-2,733	0.0261-0.0347	0.0236-0.0253	0.0205-0.0266	23-38x

R5Q5 — Cost of the growing exclusion (deduplication) condition

R5Q5: 1) The exclusion check has O(log|C|+log max{1,e}) depth and O(e log|C|) gates using a parallel comparator tree (Lemma IV.2, p.9) for e excluded indices.

As e grows from 0 to 16 in a reference build with |C|=32, the whole oracle has depth 426 with 656 logical operations at e=0, and depth 445 with 1,162 at e=16, counting each arbitrary-control operation as one logical operation. The exclusion term itself is empty at e=0, which is why the depth bound is written with max{1,e}.

2) The threshold-marking oracle is reused (Sec. IV-B, p.8); only the exclusion condition is updated each round (Sec. IV-C, p.8), so no full recompilation is needed. 3) Each verified candidate added to the exclusion condition is no longer marked, so the remaining marked count decreases by one; only verified positives update the exclusion set (Sec. IV-C, pp.8-9; Footnote 28, p.19 [35]). If the count is not known exactly, QIL determines or upper-bounds it using quantum counting or a doubling-style budget search (Lemma IV.2, p.9).